

EXAMEN FINAL TEÓRICO - 2026/1

Curso: Estructuras de datos avanzadas

Grupo: CCOMP6

Profesores: Erick Gomez Nieto / Rosa Paccotacya Yanque

Pregunta 1 (Outcomes 1,2,6)(10 puntos):

En el Baile de Navidad de Hogwarts, una tradición mágica se ha añadido este año: ¡la distribución de Galletas Encantadas como regalos entre los estudiantes! Cada galleta encantada es única y proporciona una deliciosa sorpresa mágica. Se te ha encomendado la tarea de distribuir de manera justa todas las bolsas de galletas encantadas entre los estudiantes presentes en el baile. Se te proporciona un arreglo de enteros `cookies`, donde `cookies[i]` representa la cantidad de galletas en la *i*-ésima bolsa. Además, tienes un entero *k*, que indica el número exacto de estudiantes a los que se deben distribuir todas las bolsas de galletas. Cada bolsa de galletas debe entregarse completa a un solo estudiante y no se pueden dividir entre varios.

Tu objetivo es encontrar la distribución más equitativa posible de las galletas encantadas entre los estudiantes en el Baile de Navidad de Hogwarts. Para eso debes escribir en C++ la función `distributeCookies` que dado un vector de números enteros positivos, retorne la mínima cantidad máxima de galletas que un estudiante puede recibir.

```
int distributeCookies(vector<int>& cookies, int k){  
    // Tu codigo va aqui  
}
```

Ejemplo 1 :

Input: `cookies = [8,15,10,20,8]`, `k = 2`

Output: 31

Explicación: Se puede dividir equitativamente como `[8, 15, 8]` y `[10,20]`.

- El 1er estudiante recibe `[8,15,8]` que tiene un total de $8 + 15 + 8 = 31$ galletas.

- El 2do estudiante recibe `[10,20]` lo que tiene un total de $10 + 20 = 30$ galletas.

Entonces, $\max(31,30) = 31$.

Ejemplo 2 :

Input: `cookies = [6,1,3,2,2,4,1,2]`, `k = 3`

Output: 7

Explicación: Se puede dividir equitativamente como `[6,1]`, `[3,2,2]` y `[4,1,2]`.

Entonces, $\max(7,7,7) = 7$.

Completa la implementación de la función en el archivo brindado.

El ejercicio tendrá una nota de 0 en caso de que el código entregado no compile.

Pregunta 2 (Outcomes 1,2,6)(10 puntos):

Escriba el código para implementar los algoritmos de clustering Kmeans y DBScan, y proponga una función que nos permita encontrar buenos/optimos valores de nuestros parámetros de entrada para mejorar el rendimiento del algoritmo DBScan.

(3 puntos) Funcion(es) para ejecutar Kmeans CORRECTAMENTE desde la función main()

(4 puntos) Funcion(es) para ejecutar DBScan CORRECTAMENTE desde la función main()

(3 puntos) Función de pre-procesamiento para DBScan, funcionando y CORRECTAMENTE justificada/explicada con comentarios.

Observaciones:

- El código debe ser probado con al menos 50 puntos aleatorios. Puede usar la siguiente función para generar los puntos aleatorios

```
#include <cmath>

using namespace std;

float randomFloat()
{
    return (float)(rand()) / (float)(rand());
};
```